START HERE / YOUR STEP-BY-STEP WALKTHROUGH

Your midterm, made concrete.

The interactive is built. Your next job is to understand it, test it and publish the assignment copy. This guide tells you exactly what to do.

First click

https://iron-grid-nyc.wrylake10.chatgpt.site

Open the demo. Click Take the 30-second tour. Then choose Queens, switch to report count and try a wet-day shortcut.

What you have

Deliverable	Purpose
Live Sites demo	Try the publicly deployed browser prototype immediately.
Iron_Grid_Project.zip	Website, original cleaned CSVs, preparation scripts, manifest, audit and tests.
Submission Draft PDF	Concept, EDA figures, onboarding, ethics, reflection and rubric evidence.
This walkthrough	Run, host, test, explain and refine the project.

Software you need

Publish: browser and a free GitHub account. **Edit:** VS Code is recommended. **Preview locally:** Python 3 is convenient. **Reprocess:** Python with pandas, NumPy and Matplotlib. Node.js is optional for the checks. No paid tool, Tableau license, API key or npm build is needed for the delivered website.

Extended deadline supplied by you: Wednesday, October 28, 2026.

1. Open the project on your computer.

Download and extract

Download Iron_Grid_Project.zip. In Windows, right-click and choose Extract All. Open the extracted iron-grid folder. The website to publish is inside **dist**.

Location	Purpose
dist/index.html	Page structure and onboarding dialog
dist/styles.css	Colors, spacing, responsive layout and focus styles
dist/app.mjs	D3 map, zoom, brush, linked views and downloads
dist/logic.mjs	Filtering, aggregation, ratios and date windows
dist/data/	Prepared data, enriched reports, audit and manifest
dist/vendor/	Local D3 library and license
scripts/ + source_data/	Preparation, original CSVs and validation

Run locally (optional if the live demo is enough)

1. Open **dist** in File Explorer.
2. Click its address bar, type **powershell**, and press Enter.
3. Run **py -m http.server 8000**. If py is unavailable but Python is installed, try **python -m http.server 8000**.
4. Open **http://localhost:8000** in your browser.
5. Keep the terminal open while viewing. Ctrl+C stops it.

Do not double-click index.html. Browsers can block JSON requests from file://. If neither Python command exists, deploy to GitHub Pages first or install Python 3 from its official site.

Make a change you can explain

Open the folder in VS Code. Revise one sentence in dist/index.html to match your voice, then reload. Read Data & method and explain aloud why a request is not the same thing as a failed basin.

2. Publish it on GitHub Pages.

The assignment names GitHub Pages or Observable. GitHub Pages fits this multi-file static project. A branch and root-folder publishing setup needs no build process. [A]

Create a repository

1. Sign in to GitHub; choose **New repository**.
2. Name it **iron-grid**.
3. Set it to **Public** and initialize a README.
4. Create the repository.

Upload the website

5. Choose **Add file > Upload files**.
6. Open your extracted **dist** folder. Drag its **contents** into GitHub: index.html, app.mjs, logic.mjs, styles.css, data/ and vendor/. Include .nojekyll if visible; it is optional for this app.
7. Commit to **main**.

The detail that prevents a 404

index.html must be at the repository root. Do not upload the ZIP as the website. Do not nest the entire dist folder under the root when Pages is set to /(root). Upload what is inside dist.

Turn on Pages

8. Open **Settings > Pages**.
9. Under Source, choose **Deploy from a branch**.
10. Select **main** and **/(root)**; Save.
11. Wait for deployment; Pages will show the live link.
12. Test that exact link in a private/signed-out window. It should resemble https://YOUR-USERNAME.github.io/iron-grid/.

Keep the full ZIP as your reproducible source attachment. If you also reorganize the repository to include preparation scripts, update their paths consistently; do not mix flattened website paths with unchanged scripts that expect dist/.

[A] Official GitHub branch publishing instructions. The provided demo is on Sites; nothing has been published into your GitHub account.

3. Test the experience you will demo.

Use Chrome or Firefox. Open Developer Tools > Console to spot loading errors. Browser interaction testing was unavailable during the build, so this walkthrough is an essential remaining step.

Action	Expected result
Guide: Next, Back, Close	Three clear steps; dialog closes and focus stays usable.
Reset everything	154,212 basins; 58,965 reports; 2020-01-01 to 2026-07-28.
Choose Queens	66,387 mapped basins and 23,910 supplied reports.
Choose Clogged / flooding	Report counts change; basin inventory does not.
Reset; both dates = 2021-09-02	309 reports; basin inventory remains 154,212.
Drag across the timeline	Dates, report totals, map and rankings update.
Wet-day shortcut	Two days before through four after; clipped to coverage.
Cell > street detail > dot or asset ID	Individual records display their ID and coordinates.
Pin two cells; change dates	Both cells update under the same filters.
Download cell summary	CSV contains counts, dates, denominator and interpretation.
Resize; use Tab and Enter	Readable controls; ranked list can open cells without map clicking.

If something fails

Record the action, expected result, actual result, browser and console error. A 404 usually means a missing file/folder. A blank file:// page usually needs the local server. Send a specific failure back for a focused fix.

Automated data/logic checks passed, but do not replace this browser walkthrough.

4. Understand what the code does.

Concept	Plain explanation
Data loading	The browser reads prepared JSON snapshots, not live 311/NOAA queries.
Aggregation	Each record belongs to a 1 km² hexagon. Basin counts and report counts stay separate, then are summarized in the same cell.
Cross-filtering	One state object stores borough, type and dates. The map, counts and comparisons share it. Dates affect reports; inventory stays fixed.
Brushing	A horizontal chart selection becomes start/end dates. The full timeline remains visible, and date inputs offer the same control.
Progressive disclosure	Citywide cells > an area > individual points > source record.

The identifier issue

The uploaded CSVs both call one column UNITID, but the meanings differ. Basin values identify assets, such as CB422934. Numeric complaint values identify 311 requests. Request dates were restored using 311 unique_key. The current comparison is geographic by cell, with no direct report-to-asset link.

The important formula

Reports per 100 mapped basins = selected reports in a cell / fixed mapped basins in that cell × 100. This is a spatial count ratio, not the percent of broken drains. Multiple requests can concern one location. A zero denominator means unavailable.

What D3 contributes

D3 zoom handles map transformations; brush handles time selection; scales position/color the data; quadtree finds nearby points. Canvas draws the large point inventory; SVG draws the timeline and selection.

Where edits belong

Change the story in index.html, appearance in styles.css, interactions in app.mjs and core calculations in logic.mjs. Data preparation lives in scripts/prepare_data.py. Read the README before changing data schemas.

5. Finish with the rubric in view.

Criterion	Evidence	Finish
Concept / 20%	Hidden network and report stream	State the question and screening audience
EDA & ethics / 20%	Audit, figures, date recovery, limitations	Review sources and interpretations
Creativity / 15%	Grid-to-street reveal, rainfall context	Make a visual/copy choice of your own
Interactive / 25%	Filters, brush, zoom, inspect, compare, export	Run browser checks
Feasibility / 10%	Buildless site and bounded scope	Keep hydrology extensions separate
Presentation / 10%	PDF draft and live demo	Update final URL; reflect; rehearse

Suggested finish schedule

Oct 17-19: explore, understand the ID issue, complete browser checks.
Oct 20-22: publish GitHub Pages. Ask a classmate to find a borough and period without your help.
Oct 23-25: revise confusing labels; add 2-3 specific test observations to the reflection; finish the PDF.
Oct 26-27: freeze features, rehearse, verify links.
Oct 28: submit link, PDF and source/preparation evidence.

Features to add after the core works

First: an official NYC stormwater flood scenario, with assumptions named. **Next:** multiple rain gauges or a documented radar product. **Later:** a DEM with known resolution, date, projection and vertical datum. Elevation alone does not establish drainage adequacy.

Do not claim that the current data prove old streams explain density, newer neighborhoods have better drainage, or low-report places are better served. Those are questions for additional evidence.

6. Rehearse this short presentation.

Opening: about 30 seconds

My project is The Iron Grid. It reveals NYC catch basins and compares them with reported problems. For civil engineers and environmental planners, the aim is to identify questions and places worth investigating. Undercurrents appears in the hidden drainage network and the less visible stream of public reports about it.

Live demo: about 3 minutes

0:30-1:00: Show onboarding and the equal-area citywide view.
1:00-1:30: Choose Queens. Switch basin count to report count.
1:30-2:15: Use a wet-day shortcut or brush. Show report totals updating while inventory stays fixed.
2:15-3:00: Open a cell, zoom to street detail, inspect an ID and compare two cells.
3:00-3:30: Open Data & method. Explain one limitation.

Likely questions

Why rain? It adds temporal environmental context and a meaningful linked interaction.
Why not elevation immediately? A defensible layer needs a documented DEM/datum; elevation alone does not determine capacity.
Are requests tied to a specific basin? No. They are compared spatially, and their identifiers differ.
What would an engineer do? Frame an investigation question, then verify inlet condition, grades and downstream constraints.
What is unfinished? Browser/user testing, final GitHub Pages handoff and further spatial context.

Final handoff

Submit the public GitHub Pages URL and revised PDF. Attach or link the original cleaned data, preparation code and audit. Review and customize the implementation enough to explain it accurately. If describing AI assistance, state truthfully what you reviewed and changed; the Gallery also asks how creators interpreted or modified AI outputs.

The City Gallery application is separate from the course hand-in. This package has not sent an application.